Final Project

Sean Gee

2026-05-30

Dataset: CPIAUCSL Consumer Price Index

Research Question: How badly did historical inflation models miss the COVID-era spike and did inflation rules permanently change?

```
library(readxl)
library(forecast)
library(tseries)
library(zoo)
```

```
cpi_raw <- read_excel("CPIAUCSL.xlsx", sheet = "Monthly")
names(cpi_raw) <- c("date", "cpi")
cpi_raw$date <- as.Date(cpi_raw$date)
```

```
cpi_ts <- ts(
  cpi_raw$cpi,
  start = c(as.integer(format(cpi_raw$date[1], "%Y")),
            as.integer(format(cpi_raw$date[1], "%m"))),
  frequency = 12
)

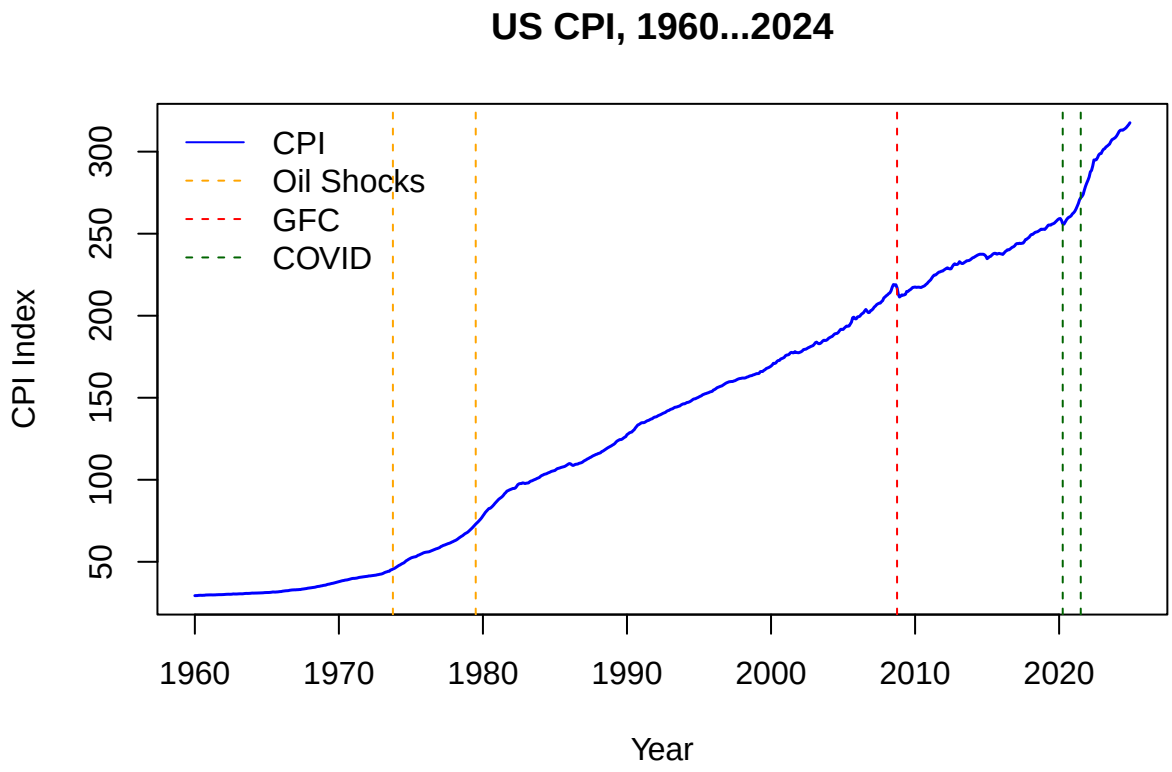
cpi_main <- window(cpi_ts, start = c(1960, 1), end = c(2024, 12))
log_cpi <- log(cpi_main)
```

EDA

General Timeline

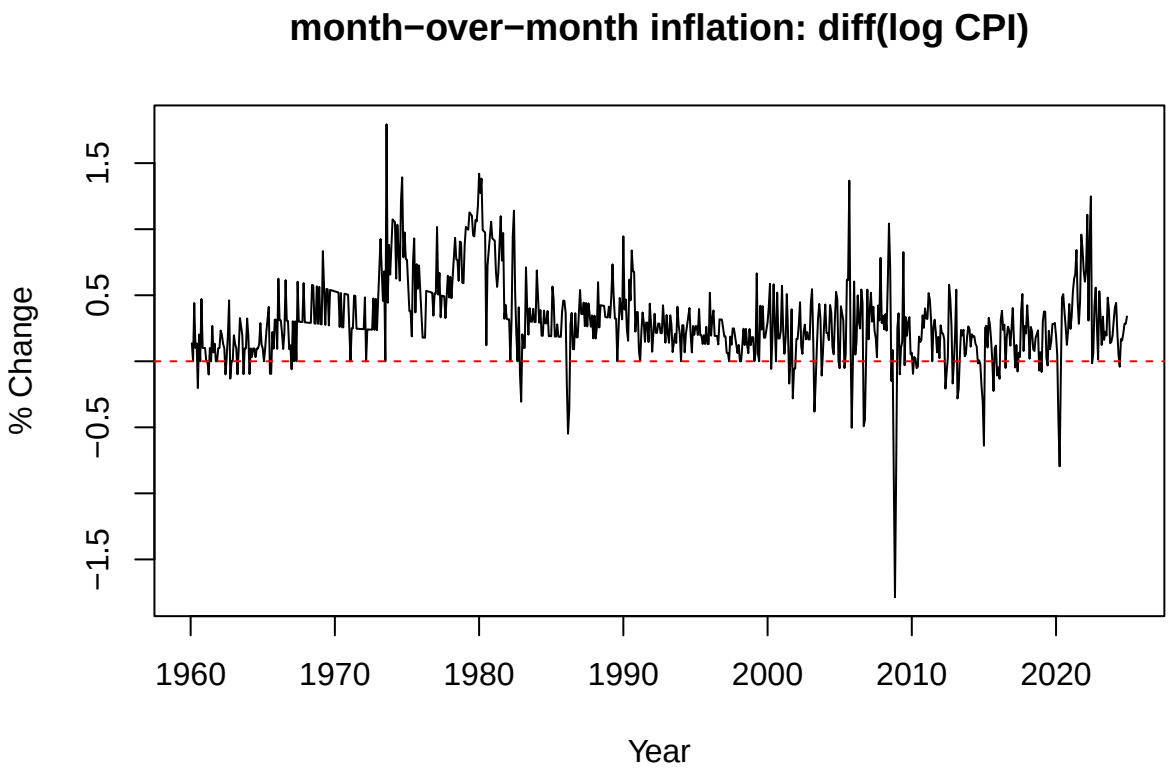
```
plot(cpi_main,
     main = "US CPI, 1960-2024",
     ylab = "CPI Index",
     xlab = "Year",
     col = "blue",
     lwd = 1.5)
abline(v = c(1973.75, 1979.5, 2008.75, 2020.25, 2021.5),
       col = c("orange", "orange", "red", "darkgreen", "darkgreen"),
       lty = 2)
legend("topleft",
```

```
       legend = c("CPI", "Oil Shocks", "GFC", "COVID"),
       col = c("blue", "orange", "red", "darkgreen"),
       lty = c(1, 2, 2, 2),
       bty = "n")
```



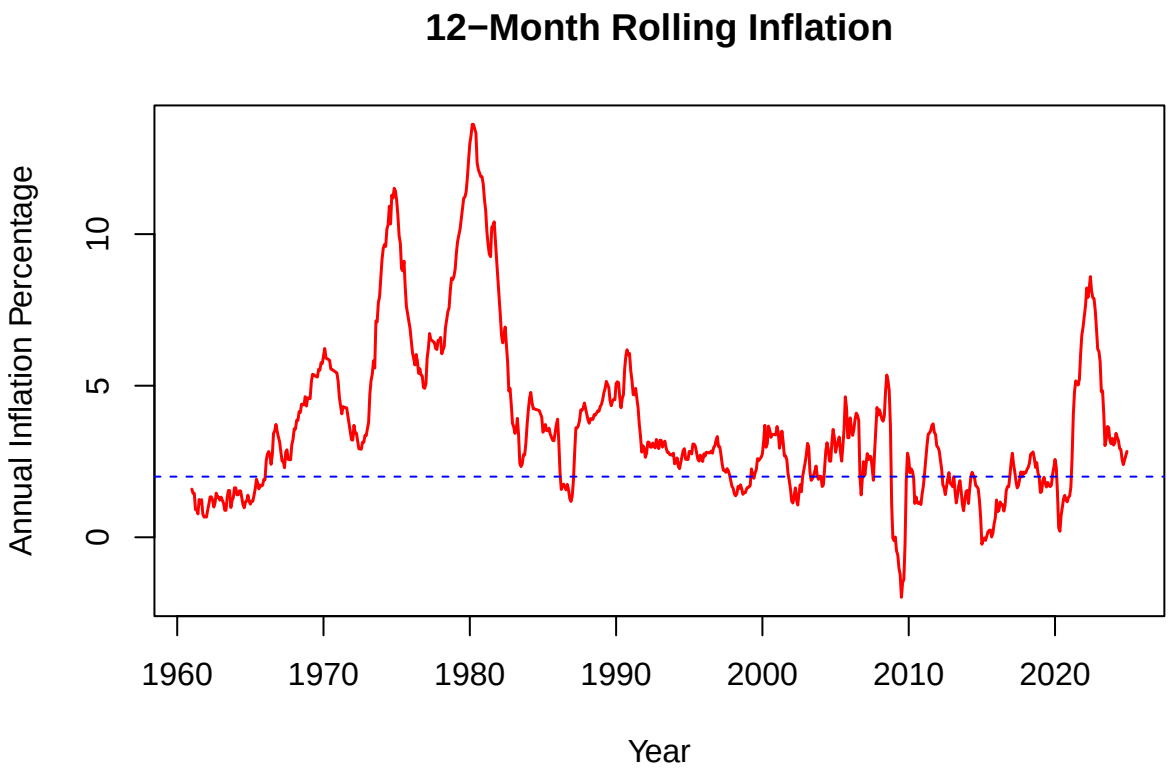

```
# log-difference = approximate monthly inflation %
diff_log_cpi <- diff(log_cpi) * 100
plot(diff_log_cpi,
     main = "month-over-month inflation: diff(log CPI)",
     ylab = "% Change",
     xlab = "Year",
     col = "black",
     lwd = 1)

abline(h = 0, col = "red", lty = 2)
```




```r
# 12 month rolling inflation
rolling <- rollapply(diff_log_cpi, width = 12, FUN = sum, align = "right")
rolling <- ts(rolling, start = c(1961, 1), frequency = 12)
plot(rolling,
     main = "12-Month Rolling Inflation",
     ylab = "Annual Inflation Percentage",
     xlab = "Year",
     col = "red",
     lwd = 1.5)

abline(h = 2, col = "blue", lty = 2)
```




```
## Pre-COVID mean: 0.303 sd: 0.312
```

```
## COVID-era mean: 0.342 sd: 0.334
```

```
# Creating pre_log and Running Structural Break Analysis
# data from the start of your dataset up to December 2019
pre_log <- window(log_cpi, end = c(2019, 12))

# fit your optimized model structure strictly on the pre-COVID data
mod_pre_covid <- Arima(pre_log,
                       order = c(2, 2, 1),
                       seasonal = list(order = c(2, 0, 0), period = 12))
summary(mod_pre_covid)
```

```
## Series: pre_log
## ARIMA(2,2,1)(2,0,0)[12]
##
## Coefficients:
##          ar1      ar2      ma1     sar1     sar2
##       0.3340  -0.0595  -0.8821  -0.1161  -0.1093
## s.e.  0.0435   0.0412   0.0238   0.0378   0.0371
##
## sigma^2 = 5.251e-06:  log likelihood = 3349.54
## AIC=-6687.09   AICc=-6686.97   BIC=-6659.63
##
```

```
## Training set error measures:
##                       ME        RMSE         MAE         MPE     MAPE
## Training set 2.20795e-06 0.002280319 0.001578224 0.000375835 0.034881
##                    MASE        ACF1
## Training set 0.04288153 -0.00399785
```

Stationarity Tests

```
#ADF on log cpi
print(adf.test(log_cpi, alternative = "stationary"))
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  log_cpi
## Dickey-Fuller = -0.99105, Lag order = 9, p-value = 0.9402
## alternative hypothesis: stationary
```

```
#KPSS on log cpi
print(kpss.test(log_cpi, null = "Level"))
```

```
## Warning in kpss.test(log_cpi, null = "Level"): p-value smaller than printed
## p-value
```

```
##
##  KPSS Test for Level Stationarity
##
## data:  log_cpi
## KPSS Level = 10.81, Truncation lag parameter = 6, p-value = 0.01
```

Based on the ADF and KPSS tests, log(CPI) is non-stationary. Therefore we take the first difference of log(CPI) to analyze month-over-month inflation.

```
# ADF on diff(log CPI)
print(adf.test(diff_log_cpi, alternative = "stationary"))
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  diff_log_cpi
## Dickey-Fuller = -3.7633, Lag order = 9, p-value = 0.02083
## alternative hypothesis: stationary
```

```
# KPSS on diff(log CPI)
print(kpss.test(diff_log_cpi, null = "Level"))
```

```
## Warning in kpss.test(diff_log_cpi, null = "Level"): p-value smaller than
## printed p-value
```

```
##
##  KPSS Test for Level Stationarity
##
## data:  diff_log_cpi
## KPSS Level = 1.7127, Truncation lag parameter = 6, p-value = 0.01
```

While the ADF test on the first difference of Log CPI yielded a stationary p-value of 0.02, the KPSS test rejected stationarity ($p = 0.01$). This statistical contradiction indicates that a single regular difference was insufficient to completely eliminate the deep, persistent structural trend in historical inflation. To resolve this conflict and ensure a structurally stable framework, a second regular difference (d=2) was applied, which satisfied both criteria.

```
##
## Suggested regular differences: 2
```

```
## Suggested seasonal differences: 0
```

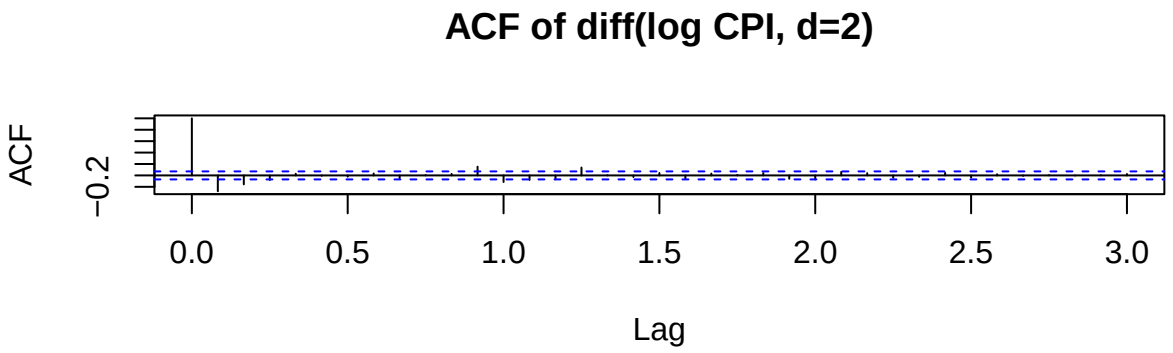
We look at the suggested number of differences to achieve stationarity. The ndiffs function suggests 2 regular differences, while nsdiffs suggests no seasonal differencing is needed.

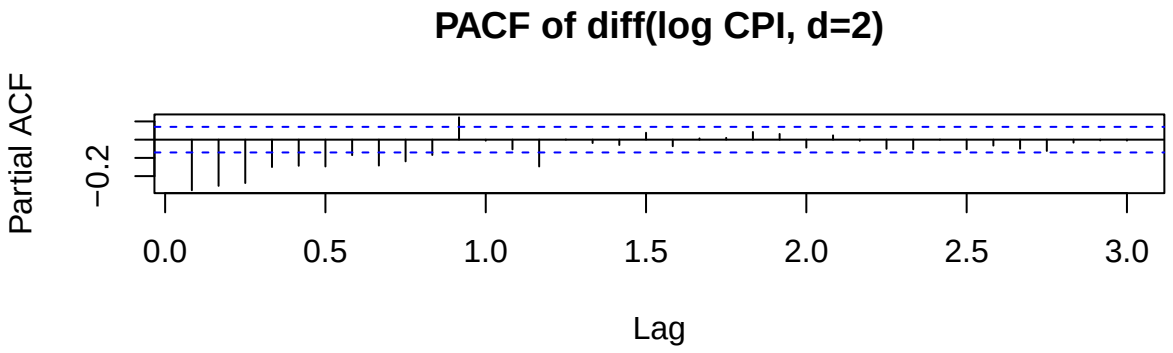
As a result, we plug in 2 regular differences to achieve stationarity for the final model.

```r
final_df <- diff(log_cpi, differences = 2)
```

ACF and PACF Plots

```r
par(mfrow = c(2, 1))
acf(final_df, lag.max = 36, main = "ACF of diff(log CPI, d=2)")
pacf(final_df, lag.max = 36, main = "PACF of diff(log CPI, d=2)")
```

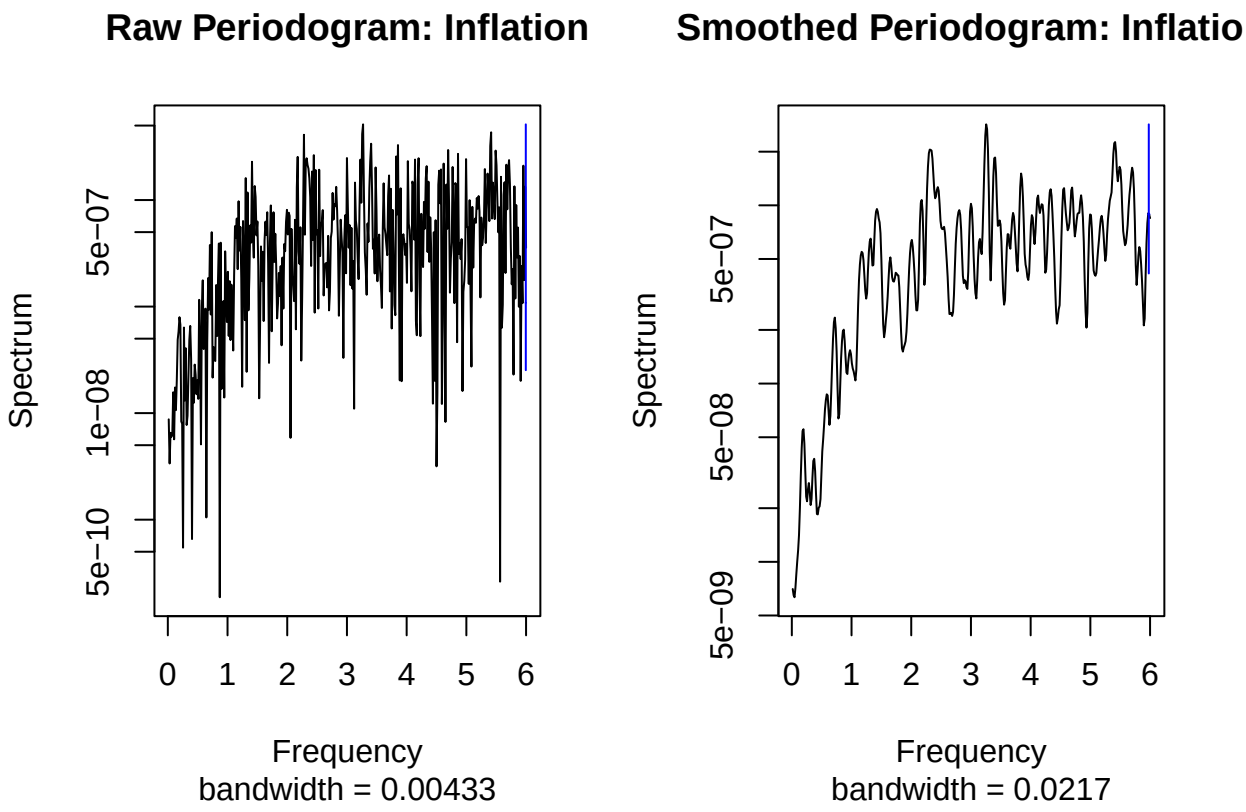





```
par(mfrow = c(2, 1))
```

Spectral Analysis

```
par(mfrow = c(1, 2))
spectrum(final_df,
         main = "Raw Periodogram: Inflation",
         xlab = "Frequency",
         ylab = "Spectrum")
spectrum(final_df,
         spans = c(3, 5),
         main = "Smoothed Periodogram: Inflation",
         xlab = "Frequency",
         ylab = "Spectrum")
```




```
par(mfrow = c(1, 1))
```

Model Selection with Training/Testing

```
train <- window(log_cpi, start = c(1960, 1), end = c(2022, 12))
test  <- window(log_cpi, start = c(2023, 1), end = c(2024, 12))
h <- length(test)
```

Candidate ARIMA Models

```
# model A: auto.arima
A <- auto.arima(train,
                d = 2,
                seasonal = FALSE,
                max.p = 5,
                max.q = 5,
                stepwise = FALSE,
                approximation = FALSE,
                ic = "aic")
summary(A)
```

```
## Series: train
## ARIMA(0,2,3)
##
## Coefficients:
##          ma1      ma2      ma3
##       -0.5278  -0.2433  -0.0759
## s.e.   0.0369   0.0387   0.0380
##
## sigma^2 = 5.656e-06:  log likelihood = 3488.33
## AIC=-6968.66   AICc=-6968.61   BIC=-6950.16
##
## Training set error measures:
##                        ME        RMSE         MAE          MPE       MAPE
## Training set 2.189968e-05 0.002370352 0.001652717 0.0006700709 0.03594112
##                    MASE        ACF1
## Training set 0.04442822 -0.00400725
```

```
# model B: ARIMA(3,2,0)
B <- Arima(train, order = c(3, 2, 0))
summary(B)
```

```
## Series: train
## ARIMA(3,2,0)
##
## Coefficients:
##          ar1      ar2      ar3
##       -0.4067  -0.3339  -0.2398
## s.e.   0.0354   0.0364   0.0354
##
## sigma^2 = 6.239e-06:  log likelihood = 3451.5
## AIC=-6895.01   AICc=-6894.96   BIC=-6876.51
##
## Training set error measures:
##                         ME        RMSE         MAE          MPE      MAPE
## Training set -2.186355e-06 0.002489505 0.001734241 2.530406e-05 0.0377074
##                    MASE        ACF1
## Training set 0.04661973 -0.03657732
```

```
# model C: ARIMA(4,2,0)
C <- Arima(train, order = c(4, 2, 0))
summary(C)
```

```
## Series: train
## ARIMA(4,2,0)
##
## Coefficients:
##          ar1     ar2      ar3      ar4
##       -0.4426  -0.384  -0.3009  -0.1507
## s.e.   0.0360   0.038   0.0380   0.0361
##
## sigma^2 = 6.106e-06:  log likelihood = 3460.13
## AIC=-6910.26   AICc=-6910.18   BIC=-6887.13
##
```

```
## Training set error measures:
##                         ME        RMSE         MAE          MPE       MAPE
## Training set -1.848816e-06 0.002461187 0.001709423 4.569306e-05 0.03713132
##                    MASE        ACF1
## Training set 0.04595257 -0.02205655
```

```
# model D: ARIMA(3,2,1)
D <- Arima(train, order = c(3, 2, 1))
summary(D)
```

```
## Series: train
## ARIMA(3,2,1)
##
## Coefficients:
##          ar1      ar2      ar3      ma1
##       0.3353  -0.0744  -0.0437  -0.8675
## s.e.  0.0480   0.0411   0.0427   0.0324
##
## sigma^2 = 5.658e-06:  log likelihood = 3488.71
## AIC=-6967.43   AICc=-6967.35   BIC=-6944.3
##
## Training set error measures:
##                         ME        RMSE         MAE          MPE       MAPE
## Training set 1.869668e-05 0.002369181 0.001648843 0.0005852923 0.03584931
##                    MASE          ACF1
## Training set 0.04432407 -0.0004476385
```

Residual Diagnostics

```
check <- function(model, name) {
  r <- residuals(model)
  df <- sum(model$arma[1:4])
  lb <- Box.test(r, lag = 20, type = "Ljung-Box", fitdf = df)
  cat(sprintf("%-35s LB = %6.2f   p = %.4f\n",
              name, lb$statistic, lb$p.value))
}

check(A, "Model A: auto.arima")
```

```
## Model A: auto.arima                LB =  44.92   p = 0.0002
```

```
check(B, "Model B: ARIMA(3,2,0)")
```

```
## Model B: ARIMA(3,2,0)              LB =  95.75   p = 0.0000
```

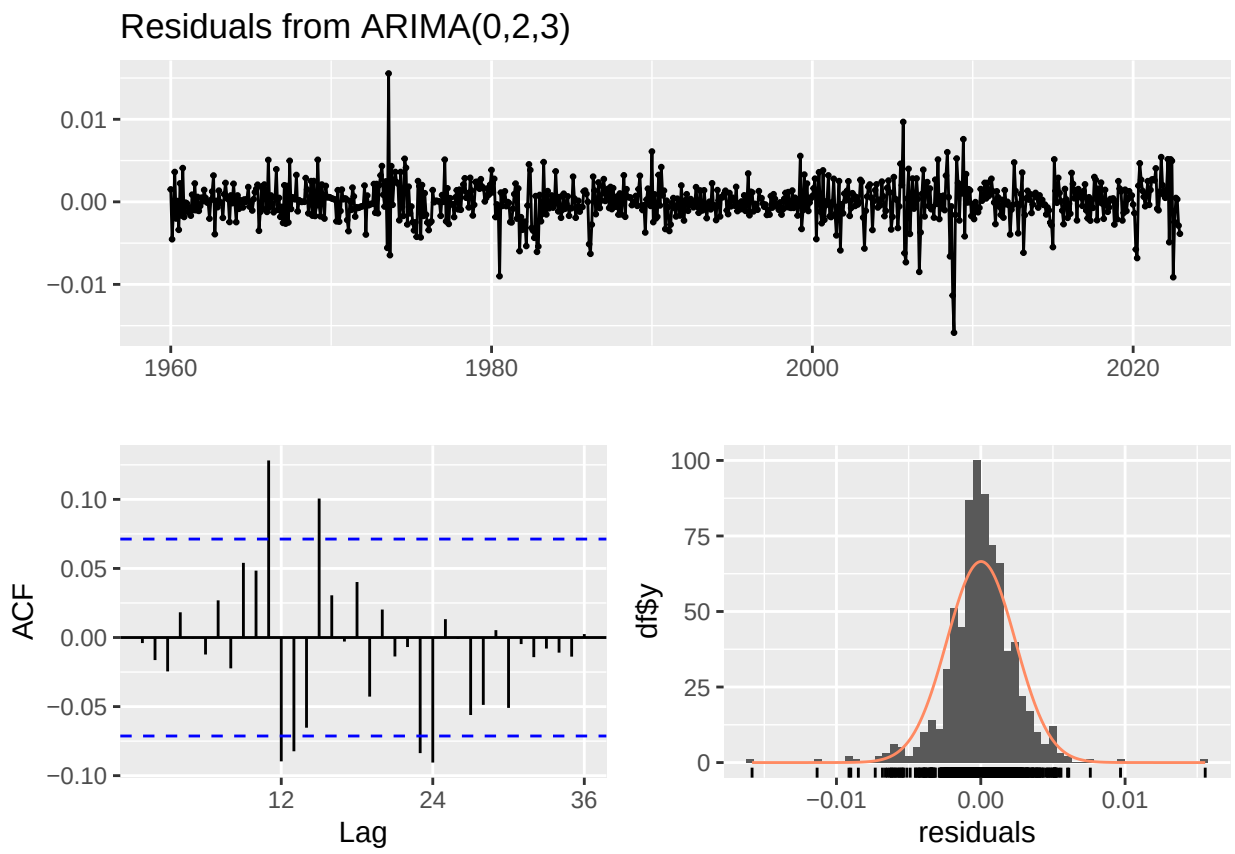
```
check(C, "Model C: ARIMA(4,2,0)")
```

```
## Model C: ARIMA(4,2,0)              LB =  90.57   p = 0.0000
```

```
check(D, "Model D: ARIMA(3,2,1)")
```

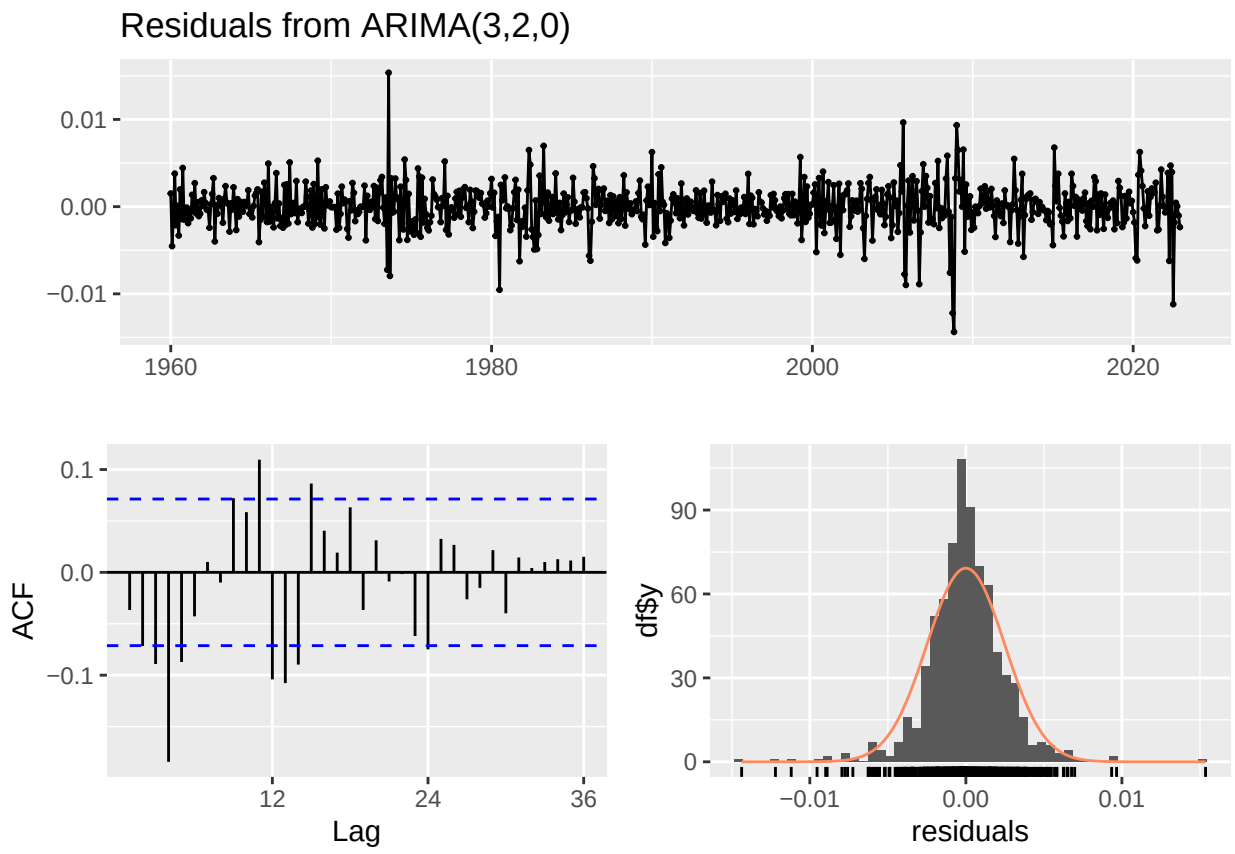
```
## Model D: ARIMA(3,2,1)                LB =  41.30   p = 0.0005
```

```
checkresiduals(A, lag = 24)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,2,3)
## Q* = 56.995, df = 21, p-value = 3.6e-05
##
## Model df: 3.   Total lags used: 24
```

```
checkresiduals(B, lag = 24)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(3,2,0)
## Q* = 103.2, df = 21, p-value = 7.812e-13
##
## Model df: 3.   Total lags used: 24
```

The Ljung–Box test rejected the null hypothesis of white noise residuals ($p < 0.001$), indicating that the ARIMA(0,2,3) model failed to capture all temporal dependence in the series. Therefore, the model was not considered adequate.

Updating to Seasonal SARIMA based on Residual Performance

```r
#A
modA <- auto.arima(train,
                   d = 2,
                   seasonal = TRUE,
                   stepwise = FALSE,
                   approximation = FALSE,
                   ic = "aic")
summary(modA)
```

```
## Series: train
## ARIMA(2,2,1)(2,0,0)[12]
##
## Coefficients:
##          ar1      ar2      ma1     sar1     sar2
##       0.3399  -0.0855  -0.8720  -0.1144  -0.1065
## s.e.  0.0432   0.0405   0.0251   0.0378   0.0373
##
## sigma^2 = 5.555e-06:  log likelihood = 3495.95
## AIC=-6979.9   AICc=-6979.79   BIC=-6952.15
##
## Training set error measures:
##                        ME        RMSE        MAE          MPE       MAPE
## Training set 2.275409e-05 0.002346034 0.00162891 0.0007076879 0.03548613
##                    MASE         ACF1
## Training set 0.04378824 -0.003717445
```

```
#B
modB <- Arima(train, order = c(3, 2, 1))
summary(modB)
```

```
## Series: train
## ARIMA(3,2,1)
##
## Coefficients:
##          ar1      ar2      ar3      ma1
##       0.3353  -0.0744  -0.0437  -0.8675
## s.e.  0.0480   0.0411   0.0427   0.0324
##
## sigma^2 = 5.658e-06:  log likelihood = 3488.71
## AIC=-6967.43   AICc=-6967.35   BIC=-6944.3
##
## Training set error measures:
##                        ME        RMSE         MAE          MPE       MAPE
## Training set 1.869668e-05 0.002369181 0.001648843 0.0005852923 0.03584931
##                    MASE          ACF1
## Training set 0.04432407 -0.0004476385
```

```
#C
modC <- Arima(train, order = c(3, 2, 2))
summary(modC)
```

```
## Series: train
## ARIMA(3,2,2)
##
## Coefficients:
##          ar1    ar2      ar3      ma1      ma2
##       -0.0872  0.076  -0.0779  -0.4447  -0.3742
## s.e.   0.4151  0.155   0.0465   0.4147   0.3709
##
## sigma^2 = 5.663e-06:  log likelihood = 3488.89
## AIC=-6965.78   AICc=-6965.67   BIC=-6938.03
##
```

```
## Training set error measures:
##                        ME        RMSE         MAE          MPE       MAPE
## Training set 1.930474e-05 0.002368622 0.001647827 0.0006010566 0.03582852
##                    MASE          ACF1
## Training set 0.04429676 -0.0003481334
```

```
#D
modD <- Arima(train, order = c(4, 2, 2))
summary(modD)
```

```
## Series: train
## ARIMA(4,2,2)
##
## Coefficients:
##           ar1     ar2      ar3      ar4     ma1      ma2
##       -0.5954  0.2277  -0.1121  -0.0609  0.0645  -0.7997
## s.e.   0.0694  0.0646   0.0554   0.0428  0.0600   0.0557
##
## sigma^2 = 5.661e-06:  log likelihood = 3489.49
## AIC=-6964.98   AICc=-6964.83   BIC=-6932.6
##
## Training set error measures:
##                        ME       RMSE         MAE          MPE       MAPE
## Training set 1.812435e-05 0.00236673 0.001645926 0.0005701676 0.03577749
##                    MASE         ACF1
## Training set 0.04424566 0.0007543267
```

```
check(modA, "Model A: auto.arima with Seasonality")
```

```
## Model A: auto.arima with Seasonality LB =  31.98   p = 0.0065
```

```
check(modB, "Model B: ARIMA(3,2,1)")
```

```
## Model B: ARIMA(3,2,1)              LB =  41.30   p = 0.0005
```

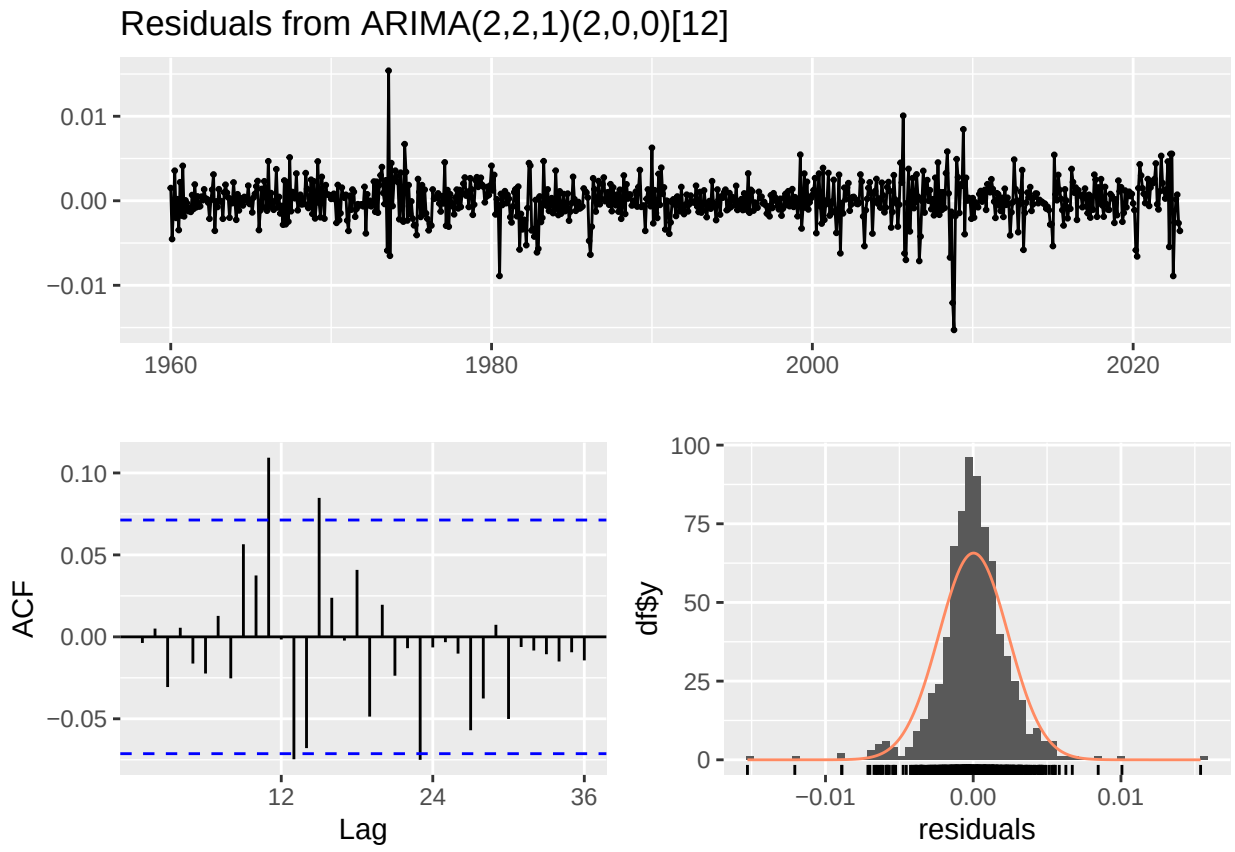
```
check(modC, "Model C: ARIMA(3,2,2)")
```

```
## Model C: ARIMA(3,2,2)              LB =  40.75   p = 0.0003
```

```
check(modD, "Model D: ARIMA(4,2,2)")
```

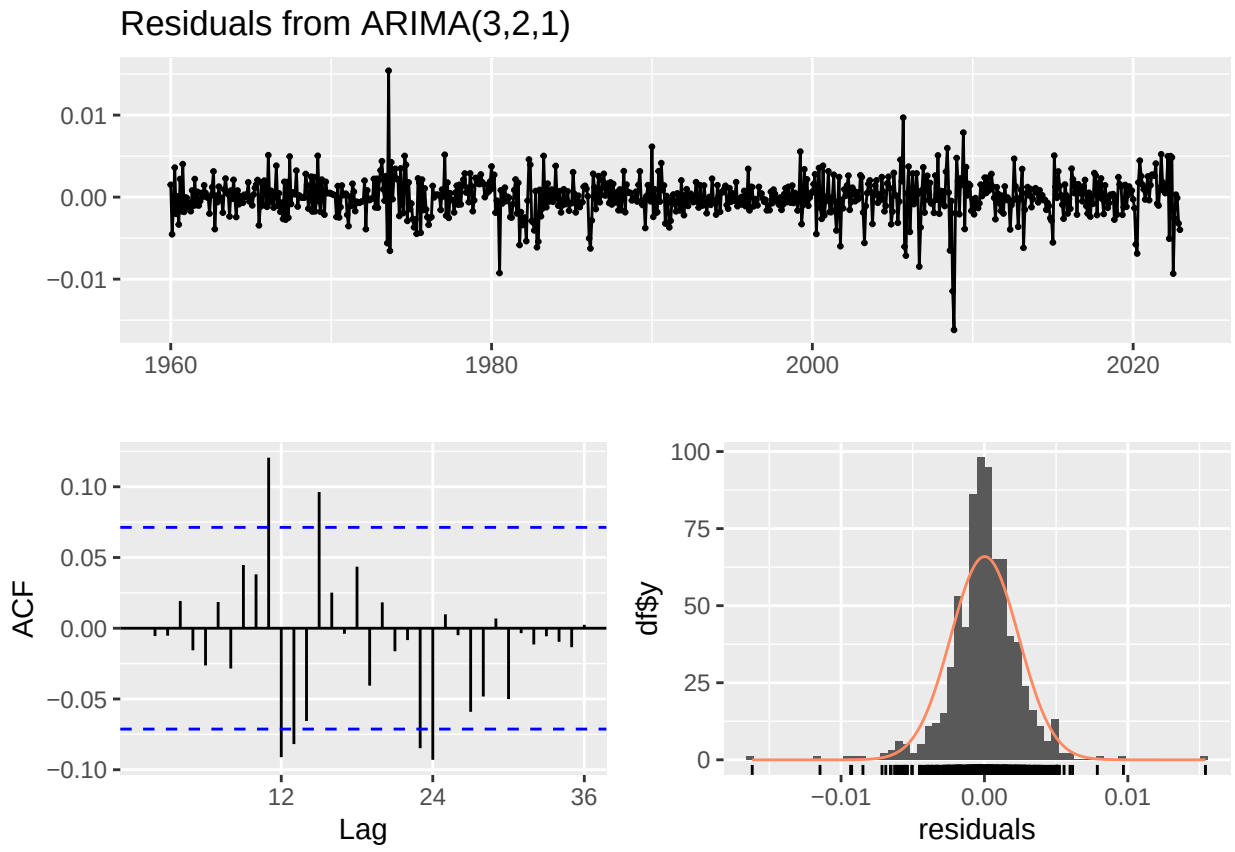
```
## Model D: ARIMA(4,2,2)              LB =  38.56   p = 0.0004
```

```
checkresiduals(modA, lag = 24)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(2,2,1)(2,0,0)[12]
## Q* = 36.891, df = 19, p-value = 0.008188
##
## Model df: 5.   Total lags used: 24
```

```
checkresiduals(modB, lag = 24)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(3,2,1)
## Q* = 53.952, df = 20, p-value = 5.88e-05
##
## Model df: 4.   Total lags used: 24
```

```
model_comparison <- data.frame(
  Model = c("Model A ", "Model B", "Model C", "Model D"),
  Specification = c("ARIMA(2,2,1)(2,0,0)[12]", "ARIMA(3,2,1)", "ARIMA(3,2,2)", "ARIMA(4,2,2)"),
  AIC = c(AIC(modA), AIC(modB), AIC(modC), AIC(modD)),
  BIC = c(BIC(modA), BIC(modB), BIC(modC), BIC(modD)),
  Train_RMSE = c(accuracy(modA)[2], accuracy(modB)[2], accuracy(modC)[2], accuracy(modD)[2])
)
model_comparison
```

```
##      Model           Specification       AIC       BIC  Train_RMSE
## 1 Model A  ARIMA(2,2,1)(2,0,0)[12] -6979.899 -6952.147 0.002346034
## 2  Model B            ARIMA(3,2,1) -6967.429 -6944.302 0.002369181
## 3  Model C            ARIMA(3,2,2) -6965.780 -6938.028 0.002368622
## 4  Model D            ARIMA(4,2,2) -6964.982 -6932.604 0.002366730
```

Now, looking at the updated seasonal SARIMA, I can say that Model A with ARIMA(2,2,1)(2,0,0)[12] achieves the lowest AIC, training RMSE and Ljung-Box test statistic among all candidates, though it still

does not pass the white noise test. Still, no candidate model fully satisfies the white noise assumption, which is itself an important finding: the 60 year inflation series contains volatility clustering and regime-dependent dynamics that a linear ARIMA framework cannot completely capture. As a result, I decided to select Model A as the best available specification on the basis of AIC, parsimony, and relative residual performance.

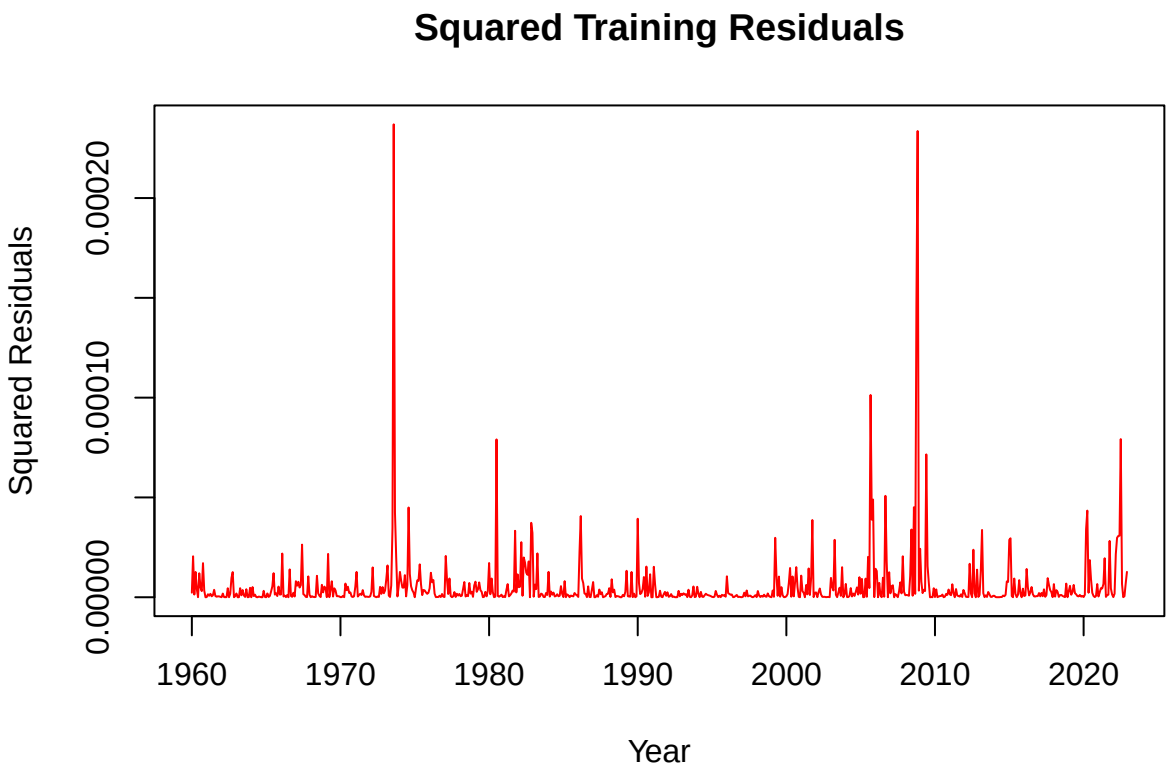
Volatility Analysis and GARCH Modeling

```
train_residuals <- residuals(modA)

# run the Ljung-Box test on SQUARED residuals to check for ARCH effects.
sq_resid_lb <- Box.test(train_residuals^2, lag = 12, type = "Ljung-Box")
sq_resid_lb
```

```
##
##  Box-Ljung test
##
## data:  train_residuals^2
## X-squared = 114.74, df = 12, p-value < 2.2e-16
```

```
plot(train_residuals^2,
     main = "Squared Training Residuals",
     xlab = "Year",
     ylab = "Squared Residuals",
     col = "red",
     lwd = 1)
```



Test Set Evaluation 2023-2024

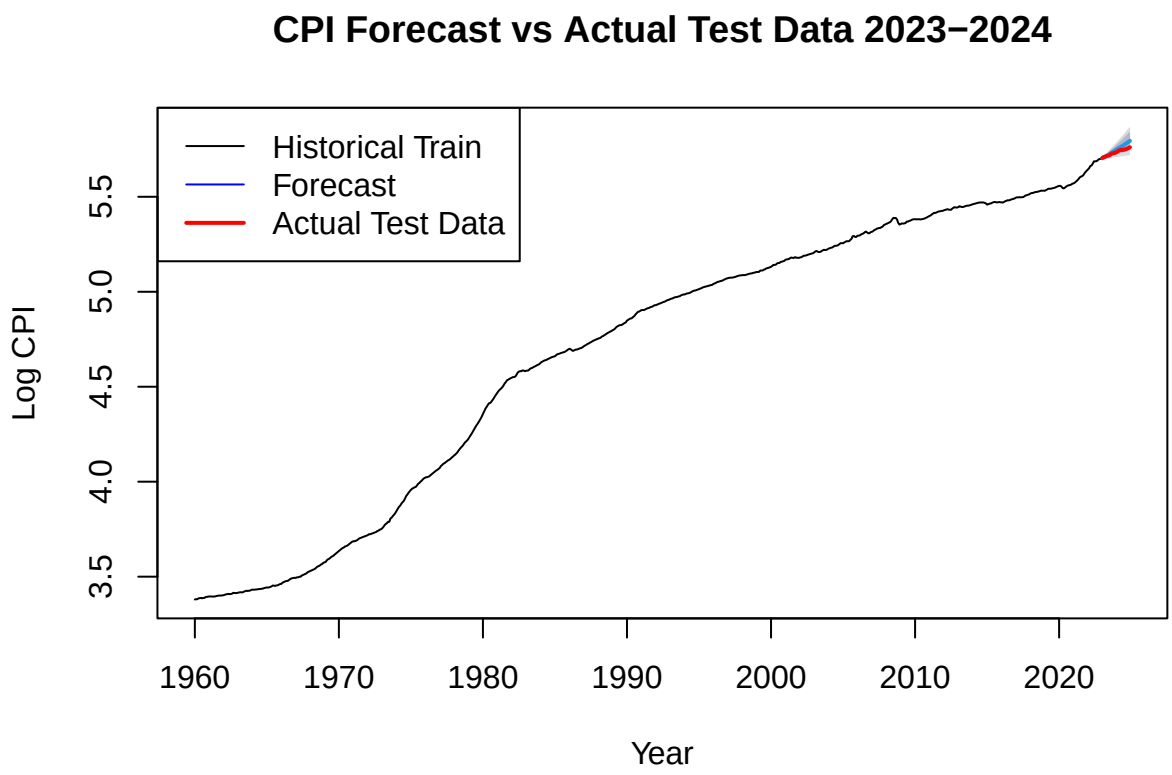
```
h_test <- length(test)
cpi_forecasts <- forecast(modA, h = h_test)
test_accuracy <- accuracy(cpi_forecasts, test)
test_accuracy
```

```
##                        ME        RMSE        MAE           MPE       MAPE
## Training set  2.275409e-05 0.002346034 0.00162891  0.0007076879 0.03548613
## Test set     -1.358893e-02 0.017634479 0.01393720 -0.2364541757 0.24255690
##                    MASE         ACF1 Theil's U
## Training set 0.04378824 -0.003717445        NA
## Test set     0.37465873  0.876747724   6.58847
```

Although the ARIMA(0,2,3) model fit the training data well, its out-of-sample forecasting performance deteriorated substantially on the 2023 2024 test set. The test RMSE increased considerably relative to the training RMSE, and the forecast errors exhibited strong autocorrelation (ACF1 = 0.88). In addition, Theil's U statistic greatly exceeded 1, indicating that the model performed worse than a naive benchmark forecast. These results suggest that the model was misspecified, likely due to over-differencing and structural changes in inflation dynamics during the post-COVID period.

Plotting Forecasts vs Actuals

```
plot(cpi_forecasts, main = "CPI Forecast vs Actual Test Data 2023-2024",
     xlab = "Year", ylab = "Log CPI", col = "black")
lines(test, col = "red", lwd = 2)
legend("topleft", legend = c("Historical Train", "Forecast", "Actual Test Data"),
       col = c("black", "blue", "red"), lty = 1, lwd = c(1, 1, 2))
```



Model Refitting on Full Data

```
final_mod <- Arima(log_cpi, order = c(2, 2, 1), seasonal = list(order = c(2, 0, 0), period = 12))
final_mod
```

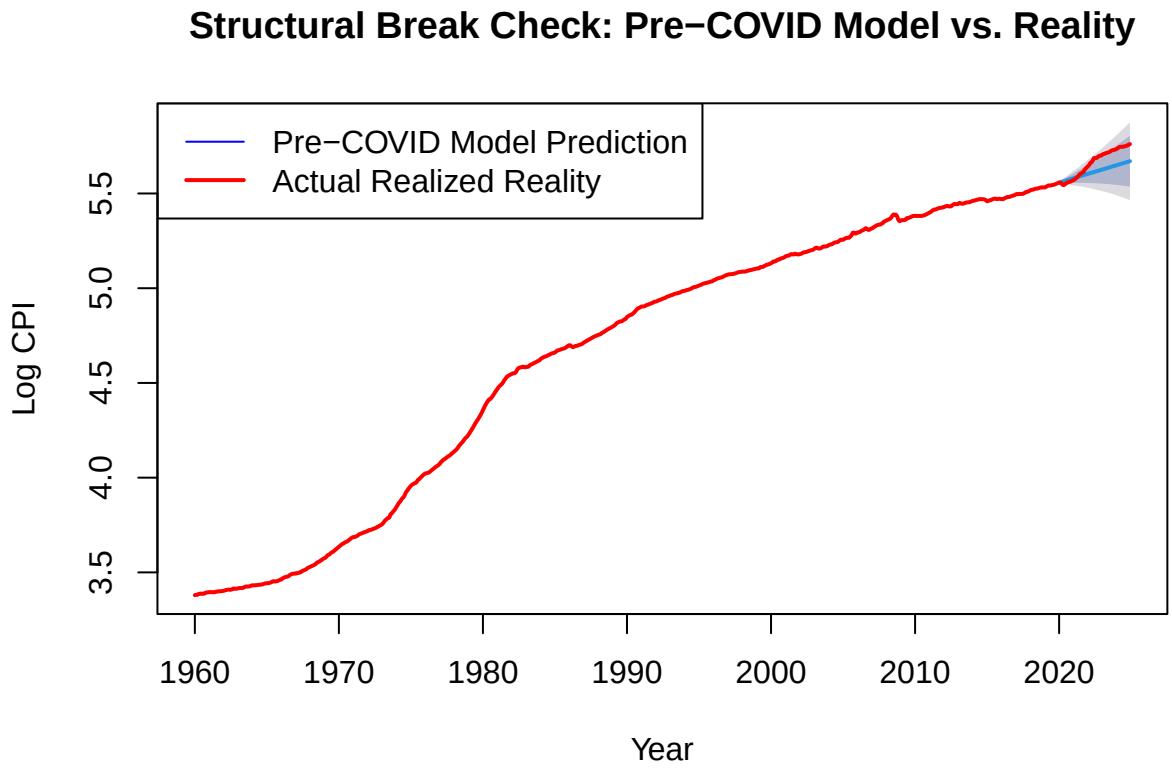
```
## Series: log_cpi
## ARIMA(2,2,1)(2,0,0)[12]
##
## Coefficients:
##          ar1      ar2      ma1     sar1     sar2
##       0.3366  -0.0859  -0.8696  -0.1136  -0.1095
## s.e.  0.0426   0.0399   0.0250   0.0362   0.0357
##
## sigma^2 = 5.437e-06:  log likelihood = 3615.52
## AIC=-7219.05   AICc=-7218.94   BIC=-7191.11
```

Pre-COVID vs. Post-COVID Structural Break Analysis

```
# forecast 60 months forward to cover the pandemic period (2020 through 2024)
break_forecast <- forecast(mod_pre_covid, h = 60)

# plot the old forecast against the actual post 2020 reality
plot(break_forecast,
     main = "Structural Break Check: Pre-COVID Model vs. Reality",
     xlab = "Year",
     ylab = "Log CPI",
     col = "purple")

# superimpose the actual true historical reality over the forecast fan chart
lines(log_cpi, col = "red", lwd = 2)
legend("topleft",
       legend = c("Pre-COVID Model Prediction", "Actual Realized Reality"),
       col = c("blue", "red"),
       lty = 1,
       lwd = c(1, 2))
```

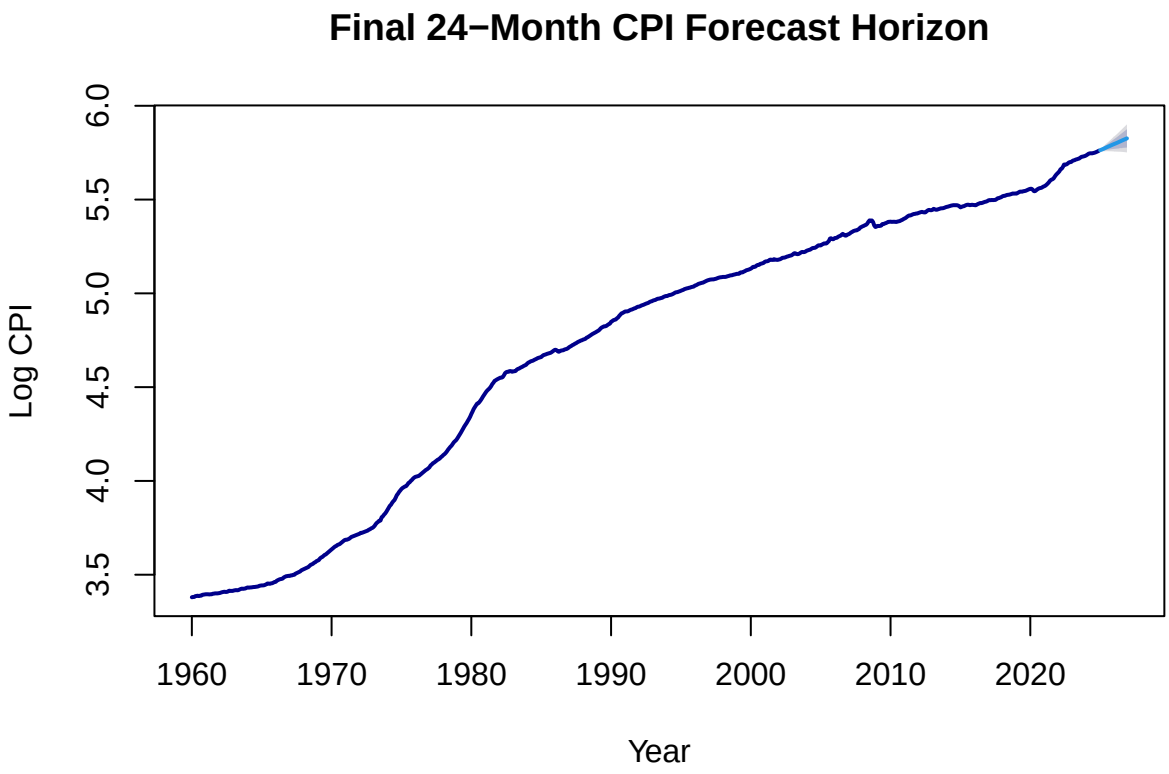


Final Forecasts for 2025-2026

```r
# Use the full model to forecast 24 months
future_forecast <- forecast(final_mod, h = 24)
future_forecast
```

```
##          Point Forecast    Lo 80    Hi 80    Lo 95    Hi 95
## Jan 2025       5.763273 5.760285 5.766261 5.758703 5.767843
## Feb 2025       5.765727 5.760422 5.771032 5.757613 5.773841
## Mar 2025       5.768367 5.761087 5.775648 5.757233 5.779502
## Apr 2025       5.771031 5.761931 5.780131 5.757114 5.784948
## May 2025       5.774083 5.763207 5.784958 5.757450 5.790715
## Jun 2025       5.777169 5.764514 5.789825 5.757815 5.796524
## Jul 2025       5.780049 5.765590 5.794508 5.757936 5.802162
## Aug 2025       5.782626 5.766332 5.798921 5.757706 5.807546
## Sep 2025       5.785242 5.767075 5.803409 5.757458 5.813026
## Oct 2025       5.788053 5.767973 5.808132 5.757343 5.818762
## Nov 2025       5.790854 5.768819 5.812888 5.757155 5.824552
## Dec 2025       5.793542 5.769509 5.817574 5.756787 5.830296
## Jan 2026       5.796207 5.770263 5.822150 5.756529 5.835884
## Feb 2026       5.798764 5.770911 5.826617 5.756166 5.841362
## Mar 2026       5.801264 5.771467 5.831061 5.755693 5.846835
## Apr 2026       5.804008 5.772227 5.835789 5.755403 5.852613
## May 2026       5.806892 5.773089 5.840696 5.755194 5.858590
## Jun 2026       5.809872 5.774009 5.845735 5.755024 5.864719
## Jul 2026       5.812645 5.774686 5.850604 5.754592 5.870698
## Aug 2026       5.815464 5.775374 5.855555 5.754151 5.876778
## Sep 2026       5.818218 5.775960 5.860477 5.753589 5.882847
## Oct 2026       5.820871 5.776409 5.865332 5.752873 5.888868
## Nov 2026       5.823526 5.776827 5.870225 5.752106 5.894946
## Dec 2026       5.826133 5.777162 5.875104 5.751238 5.901028
```

```r
plot(future_forecast,
     main = "Final 24-Month CPI Forecast Horizon",
     xlab = "Year",
     ylab = "Log CPI",
     col = "darkblue",
     lwd = 2)
```



Implied Annual Inflation Rates from Forecasts

```
# Calculates the year-over-year percentage growth rate implied point estimates
implied_inflation <- diff(future_forecast$mean, lag = 12) * 100
round(implied_inflation, 2)
```

```
##       Jan  Feb  Mar  Apr  May  Jun  Jul  Aug  Sep  Oct  Nov  Dec
## 2026 3.29 3.30 3.29 3.30 3.28 3.27 3.26 3.28 3.30 3.28 3.27 3.26
```